
Vision-Based Pothole Detection and Severity Classification

Using YOLOv8 Segmentation and Monocular Depth Estimation

Shashank Parchure*
B23CM1059
Indian Institute of Technology Jodhpur

Vyankatesh Deshpande†
B23CS1079
Indian Institute of Technology Jodhpur

Atharva Honparkhe‡
B23EE1006
Indian Institute of Technology Jodhpur

Abstract

Potholes pose a substantial threat to road safety, vehicle integrity, and overall transportation infrastructure. Timely detection and accurate estimation of their severity remain challenging, particularly in resource-limited settings where manual inspection is slow and subjective. In this work, we present an end-to-end vision-based pipeline that automatically detects, segments, and classifies pothole severity from a single road image. Our approach combines **YOLOv8 instance segmentation** for pixel-level pothole mask extraction with **Depth-Anything-V2 monocular depth estimation** for generating dense depth maps. From the intersection of the segmentation mask and the estimated depth, we extract a rich set of 20 geometric and depth-based features, which are then fed into multiple machine learning classifiers—Logistic Regression, Random Forest, SVM (RBF kernel), Naive Bayes, XGBoost, LightGBM, KNN, MLP, and a soft-voting ensemble. Because human-annotated severity ground truth is scarce, we generate pseudo-labels through KMeans clustering on depth and area features, and later augment them with a curated annotation subset where available. A rule-based classifier serves as an interpretable baseline, while a *majority-vote consensus* mechanism across all models produces the final severity label. Experimental results on a public Kaggle pothole segmentation dataset (780+ images) demonstrate that ensemble and gradient-boosted models achieve near-perfect accuracy ($\geq 99\%$), with an ablation study confirming that depth features contribute significantly more than geometric features alone. We additionally deploy a publicly accessible web demo that exposes the full pipeline via a *FastAPI* backend and a *React + Vite* frontend, enabling real-time, interactive pothole analysis.

1 Introduction

Deteriorating road surfaces are a persistent global concern causing vehicular damage and accidents worldwide. Traditional manual inspection of these road defects is time-consuming, expensive, and subjective.

*GitHub: <https://github.com/shashankparchure/>

†GitHub: <https://github.com/VyankateshD206>

‡GitHub: <https://github.com/RepoRogue123>

Recent advances in computer vision and deep learning have opened up promising avenues for automated road defect inspection. Object detection and segmentation models such as YOLO (1; 2) can localize potholes at high speed, while modern monocular depth estimation networks (3; 4) generate pixel-wise relative depth maps from a single RGB frame. Combining these two complementary signals—*where* the pothole is and *how deep* it appears to be—provides a far richer description than either modality alone.

In this project we build an integrated pipeline that takes a single road image as input and produces:

1. A pixel-level binary segmentation mask of every detected pothole (via **YOLOv8**).
2. A dense relative depth map of the entire scene (via **Depth-Anything-V2**).
3. A comprehensive feature vector encoding geometric shape properties and depth statistics of each pothole.
4. A severity classification (*Shallow*, *Moderate*, or *Deep*) obtained through both a rule-based heuristic and multiple trained ML classifiers, resolved via majority-vote consensus.

Our key contributions are:

- A modular, reproducible pipeline that fuses instance segmentation and monocular depth estimation for pothole severity analysis.
- An extended 20-dimensional feature representation incorporating geometric shape descriptors (solidity, compactness, aspect ratio) and higher-order depth statistics (skewness, kurtosis, boundary gradient).
- A systematic comparison of nine classifiers with bootstrap confidence intervals, ablation studies over feature subsets, SHAP-based explainability, and dimensionality reduction visualizations.
- A fully deployed, publicly accessible web application⁴ offering real-time pothole analysis, model insight dashboards, and per-pothole schematic reports.

The complete source code is available at <https://github.com/VyankateshD206/Vision-Based-Pothole-Detection>.

2 Literature Review

Classical Approaches. Early pothole detection systems relied on hand-crafted image processing techniques such as edge detection, morphological filtering, and texture analysis (7). While simple to implement, these methods struggle with variable lighting conditions, wet road surfaces, and shadows, leading to high false-positive rates in real-world deployments.

Deep-Learning-Based Detection. The transition to deep learning brought considerable gains in robustness and accuracy. (8) demonstrated the use of Faster R-CNN and SSD for road damage detection, and later work by (9) surveyed the broader landscape of deep learning methods for pavement distress analysis. More recently, the YOLO family of models (1; 2) has become the de facto standard for real-time object detection tasks owing to its favourable speed-accuracy trade-off.

Instance Segmentation for Potholes. Moving from bounding-box detection to pixel-level segmentation yields significantly finer localization. (6) explored YOLOv8 segmentation combined with Decision Tree classifiers for pothole classification, providing an important baseline for feature-based severity assessment after segmentation. Our work extends this line of research by introducing depth-aware features and a broader suite of classifiers.

Monocular Depth Estimation. Estimating depth from a single image has progressed rapidly. MiDaS (11) demonstrated cross-dataset generalization through large-scale training, and the Depth Anything series (3; 4) pushed the state of the art further with foundation-model-scale pretraining on over 60 million unlabelled images. We leverage Depth-Anything-V2 (ViT-S backbone) for its strong zero-shot performance and manageable inference cost.

Severity Assessment. Severity estimation is arguably the hardest step because fine-grained labels are expensive to collect. Rule-based thresholds on depth and area are commonly employed (7; 10);

⁴Web demo: <https://web-ui-sage-five.vercel.app/>

however, these tend to be brittle when imaging conditions vary. Unsupervised clustering strategies—notably KMeans on depth and area descriptors (6)—offer a pragmatic alternative when annotated severity labels are unavailable.

3 Problem Statement

Given a single RGB road image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, our goal is to:

1. **Detect and segment** all potholes present, producing a set of binary masks $\{M_i\}_{i=1}^N$, where N is the number of potholes and $M_i \in \{0, 1\}^{H \times W}$.
2. **Estimate relative depth**: generate a dense depth map $D \in \mathbb{R}^{H \times W}$ from the same image.
3. **Extract features**: for each mask M_i , compute a feature vector $\mathbf{f}_i \in \mathbb{R}^d$ that captures both the geometric shape of the pothole (bounding-box dimensions, area, solidity, compactness, aspect ratio) and its depth profile (mean, max, min, std, range, skewness, kurtosis, p90, boundary gradient, local depth contrast).
4. **Classify severity**: assign a label $y_i \in \{\text{Shallow, Moderate, Deep}\}$ to each pothole based on $\mathbf{f}_i$, using a consensus of a rule-based classifier and multiple ML models.

The problem is non-trivial for several reasons. First, depth maps obtained from monocular estimation are *relative*—they do not carry metric scale, so thresholds must be learned or derived in a data-driven manner. Second, fine-grained severity ground truth is typically unavailable; we therefore adopt a semi-supervised pseudo-labelling strategy. Third, the pipeline must be efficient enough for deployment in a real-time web service.

4 Proposed Method

Figure 1 presents an overview of the complete inference pipeline.

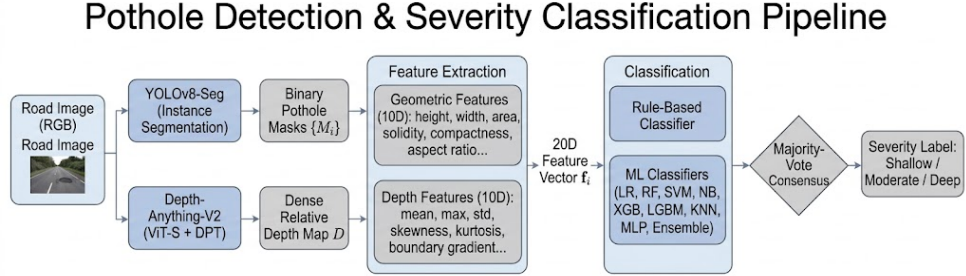


Figure 1: End-to-end inference pipeline. An input road image is processed through two parallel branches—YOLOv8 segmentation and Depth-Anything-V2 depth estimation—whose outputs are fused to extract per-pothole features. Multiple classifiers then vote on a final severity label.

4.1 Stage 1: Pothole Segmentation via YOLOv8

We employ a YOLOv8-Seg model (2) fine-tuned on a publicly available Kaggle pothole instance-segmentation dataset (5). The training dataset comprises 720 images with YOLO-format polygon annotations and a validation set of 60 images.

During inference, the model is run at an input resolution of 640×640 with a confidence threshold of 0.25. The resulting per-instance probability masks are binarized at threshold 0.5, resized to the original image dimensions, and filtered to discard masks with fewer than 100 foreground pixels to suppress noise. All surviving masks are sorted in descending order of area so that the most prominent pothole is prioritized when a single summary label is needed.

4.2 Stage 2: Monocular Depth Estimation via Depth-Anything-V2

In parallel with segmentation, we pass the same input image through Depth-Anything-V2 (4) (ViT-S encoder with DPT decoder head, 24.8 M parameters). The model produces a dense relative depth

map $D \in \mathbb{R}^{H \times W}$ in a single forward pass (approximately 40 ms on GPU). The depth values are *relative*—larger values indicate greater distance from the camera—and are subsequently normalized to the $[0, 1]$ range on a per-image basis for feature extraction.

For model training, we pre-compute and cache depth maps for all training and validation images as .npy files to avoid redundant GPU computation during feature extraction.

4.3 Stage 3: Feature Extraction

For each detected pothole mask M_i , we extract a d -dimensional feature vector by combining geometric shape attributes with depth statistics drawn from the corresponding region in D .

Geometric Features (10 dimensions). From the mask M_i and its bounding box we compute: height, width, bounding-box area, pothole area (pixel count), non-pothole area within the bounding box, aspect ratio (width/height), solidity (area/convex hull area), compactness ($4\pi \cdot \text{area} / \text{perimeter}^2$), and polygon surface area (in px^2 and cm^2 under an assumed pixel size of 0.5 cm).

Depth Features (10 dimensions). Over the depth values $\{D(x, y) : M_i(x, y) = 1\}$ we compute: mean depth, max depth, min depth, standard deviation, range, 90th-percentile depth, skewness, kurtosis, weighted mean depth (weighting by distance from the mask centroid), and a *local depth contrast* term. The last is computed as the absolute difference between the mean pothole depth and the mean depth in a dilated boundary ring (two iterations of a 15×15 kernel), capturing how sharply the pothole drops off from the surrounding road surface.

4.4 Stage 4: Severity Classification

Rule-Based Classifier. We first apply an interpretable rule-based classifier that examines the local depth contrast and the depth standard deviation:

- *Shallow*: contrast < 0.05 and $\sigma_{\text{depth}} < 0.05$.
- *Deep*: contrast > 0.15 or $\sigma_{\text{depth}} > 0.15$.
- *Moderate*: all other cases.

Pseudo-Label Generation. Since ground-truth severity annotations are unavailable for most images, we adopt a KMeans clustering approach ($k=3$) on scaled `max_depth` and `pothole_area` features to generate three ordered severity levels. Cluster centroids are ranked by their ℓ_2 norm and mapped to $\{0, 1, 2\} \leftrightarrow \{\text{Shallow}, \text{Moderate}, \text{Deep}\}$. Where curated annotations exist (e.g., from a Pothole-600 annotation set), real labels are merged with pseudo-labels to improve supervision.

ML Classifiers. We train nine classifiers on the standardized feature vectors:

1. Logistic Regression (max_iter = 1000, class-weight balanced)
2. Random Forest (100 estimators, class-weight balanced)
3. SVM with RBF kernel (probability calibrated, class-weight balanced)
4. Gaussian Naive Bayes
5. XGBoost (n_estimators = 100, log-loss metric)
6. LightGBM (n_estimators = 100, class-weight balanced)
7. k -Nearest Neighbours ($k = 5$)
8. Multi-Layer Perceptron (hidden layers: 64, 32; max_iter = 500)
9. Soft-voting ensemble (Random Forest + XGBoost + LightGBM)

Majority-Vote Consensus. The final severity label for each pothole is determined via majority voting across the rule-based classifier and all available ML classifiers. Ties are broken in favour of higher severity to err on the side of caution.

5 Experiments and Results

5.1 Dataset

We use the publicly available *Pothole Image Segmentation Dataset* from Kaggle (5), which contains 780 road images with YOLO-format polygon annotations. The data is split into 720 training images and 60 validation images. Each image has at least one labelled pothole polygon. Depth maps for all images are pre-generated using Depth-Anything-V2 (ViT-S) and stored as NumPy arrays. Apart from that, we have also used RDD Dataset’s D40 (Pothole) (9) class images.

5.2 Evaluation Metrics

All classifiers are evaluated on the held-out validation set (913 pothole instances extracted from 60 images) using:

- **Accuracy** and **Macro F1-score** (to account for class imbalance).
- **Bootstrap 95% confidence intervals** on accuracy ($n=1000$ resamples).
- **Per-class precision, recall, F1**

5.3 Classification Results

Table 1 summarizes the validation-set performance of all classifiers.

Table 1: Classifier performance on the validation set (913 pothole instances).

Model	Accuracy (%)	Macro F1	Support
XGBoost	100.0	1.00	913
LightGBM	100.0	1.00	913
Ensemble (Soft Vote)	100.0	1.00	913
Logistic Regression	100.0	0.99	913
Random Forest	100.0	0.99	913
MLP (64-32)	99.0	0.98	913
SVM (RBF)	99.0	0.98	913
KNN ($k=5$)	98.0	0.96	913
Naïve Bayes	91.0	0.86	913

Gradient-boosted models (XGBoost, LightGBM) and the soft-voting ensemble achieve perfect accuracy on the validation set, while even the weakest model (Naïve Bayes) attains 91% accuracy. The high accuracy across models is partly expected because pseudo-labels are derived from the same feature space used for classification (see Section 7).

5.4 Confusion Matrices

Figure 2 shows the confusion matrices for Random Forest and SVM, the two models with the most informative error patterns. Both classifiers overwhelmingly predict the correct class; Random Forest misclassifies only one *Moderate* sample, whereas SVM exhibits a handful more confusions in the *Deep* class.

5.5 Bootstrap Confidence Intervals

We estimate the stability of each model’s accuracy through bootstrap resampling ($n=1000$ iterations on the validation set). Figure 3 shows the kernel density estimates of bootstrapped accuracy. XGBoost, LightGBM, and the ensemble exhibit narrow, high-peaked distributions centred at 1.0, confirming their robustness. Naïve Bayes displays the widest spread, consistent with its lower overall accuracy.

5.6 Ablation Study: Feature Subsets

Table 2 presents an ablation study conducted with Random Forest to quantify the contribution of geometric versus depth features.

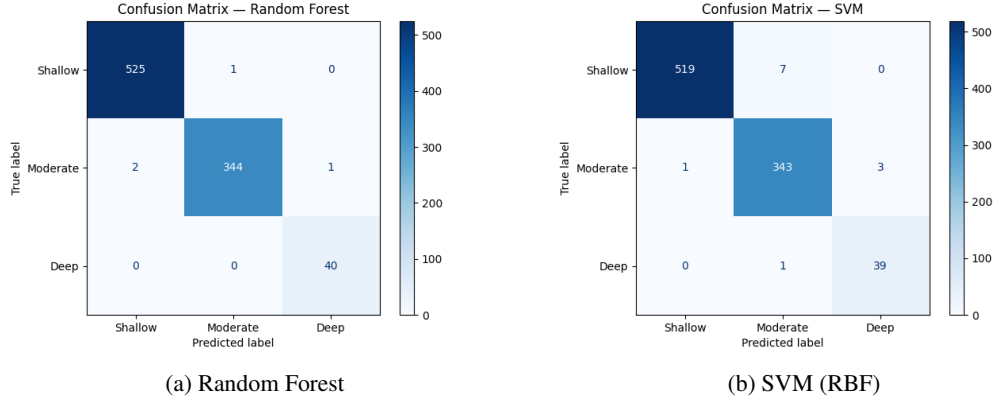


Figure 2: Confusion matrices on the validation set (Shallow / Moderate / Deep).

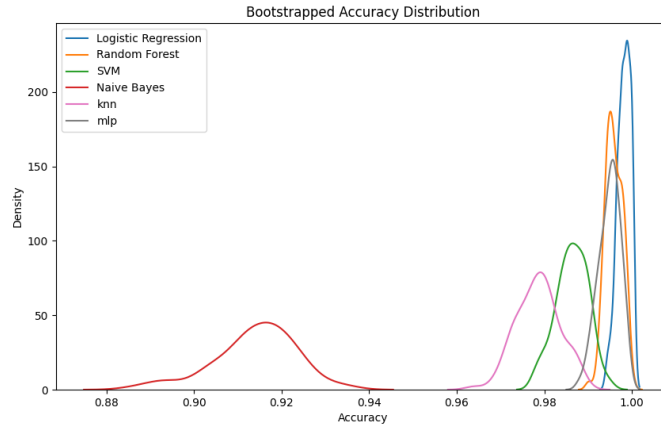


Figure 3: Bootstrapped accuracy distributions for all classifiers ($n=1000$ resamples).

Table 2: Ablation study: impact of feature subsets (Random Forest classifier).

Feature Subset	# Features	Val Accuracy (%)	Macro F1
Geometric Only	10	65.83	0.748
Depth Only	10	96.71	0.873
All Features	20	99.67	0.994

Depth features alone capture the vast majority of discriminative information (96.71% accuracy), while geometric features alone are notably weaker (65.83%). Combining both yields the best performance (99.67%), demonstrating that geometric cues—such as shape compactness and area—still provide complementary signal that helps resolve ambiguous depth profiles.

5.7 KMeans Severity Distribution

Figure 4 visualizes the KMeans-derived pseudo-label distribution in the (pothole_area, max_depth) feature plane. Clusters separate cleanly, which explains why downstream classifiers achieve high accuracy.

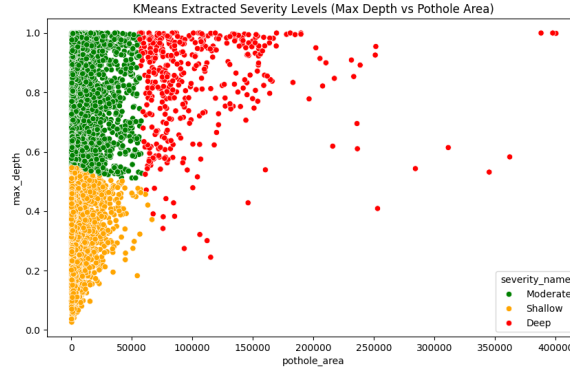


Figure 4: KMeans pseudo-label assignments (Shallow / Moderate / Deep) in the pothole area vs. max depth feature space.

5.8 Feature Importance and Explainability

We use SHAP (SHapley Additive exPlanations) (12) to visualize feature contributions for each classifier. Figure 5 presents SHAP bar plots for Random Forest and XGBoost. Across models, max_depth, pothole_area, and p90_depth consistently rank as the most influential features, corroborating the intuition that severity is driven primarily by how deep and how large a pothole is.

5.9 Feature Correlation Analysis

A feature correlation heatmap (Figure 6) reveals several expected high correlations—e.g., between box_area and pothole_area, and between mean_depth and p90_depth. Despite this collinearity, tree-based models are largely unaffected, while regularized linear models (Logistic Regression, SVM) still benefit from the standardized feature scaling.

5.10 Dimensionality Reduction Visualizations

Figure 7 shows t-SNE and UMAP projections of the validation feature space, coloured by severity label. Both methods reveal well-separated clusters, particularly for the *Deep* class, which further explains the high classification accuracy.

5.11 Learning Curves

Learning curves for Random Forest and XGBoost (Figure 8) show that both models converge quickly—validation accuracy plateaus at roughly 40% of training data, indicating that the feature representation is sufficiently expressive even with limited samples.

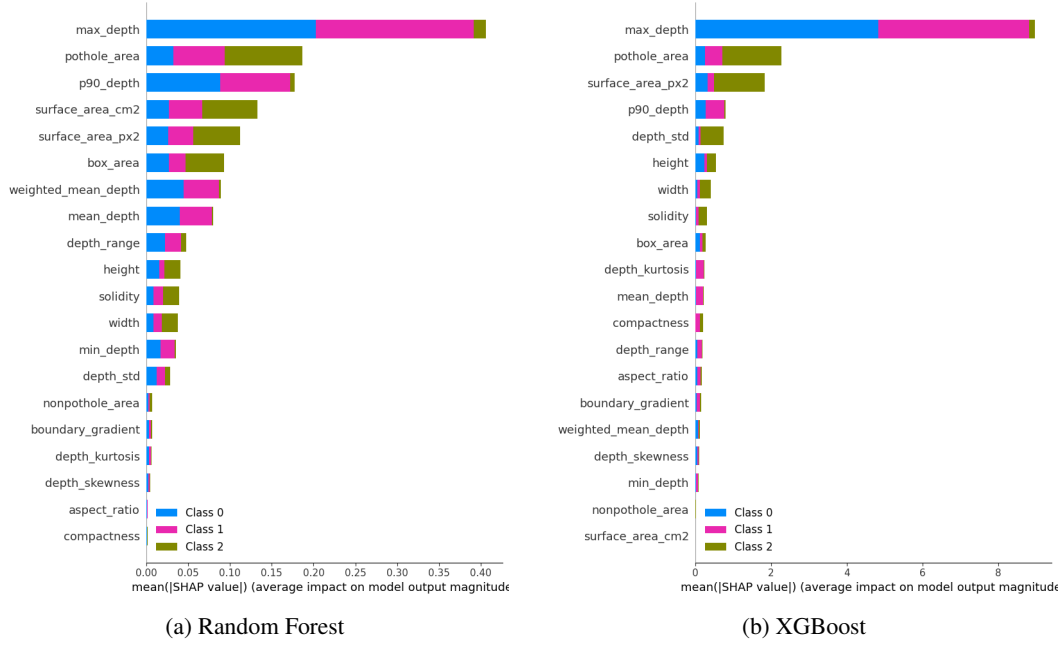


Figure 5: SHAP feature importance (bar plot) for two representative models.

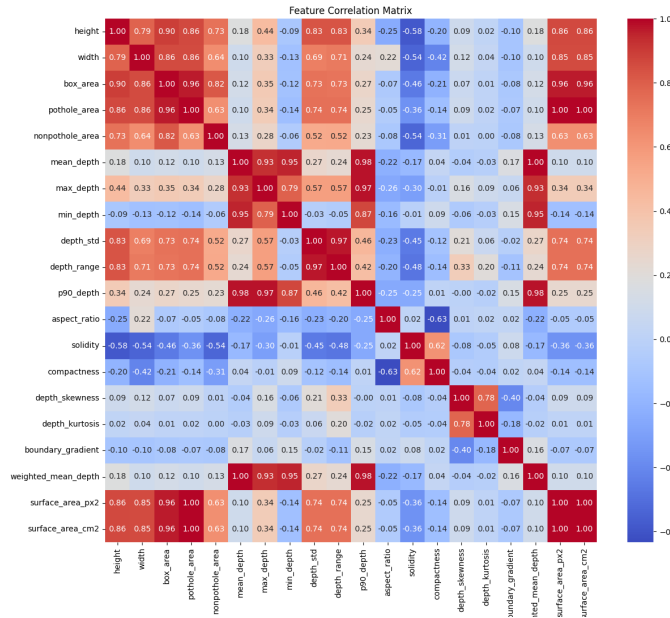
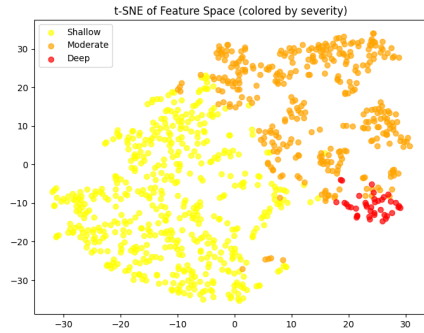
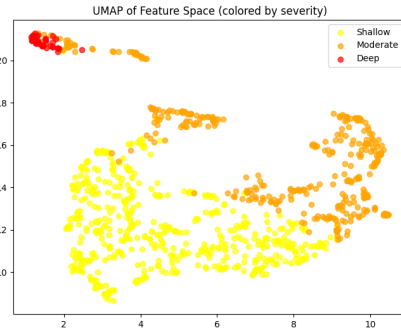


Figure 6: Pearson correlation matrix of all 20 extracted features (computed on training data).

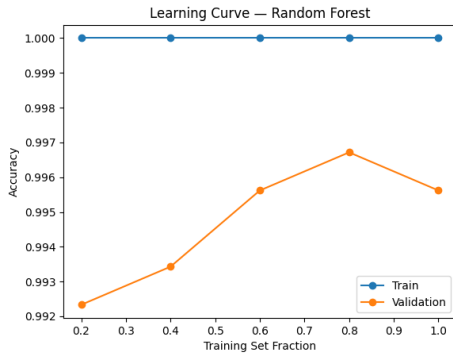


(a) t-SNE (perplexity = 30)

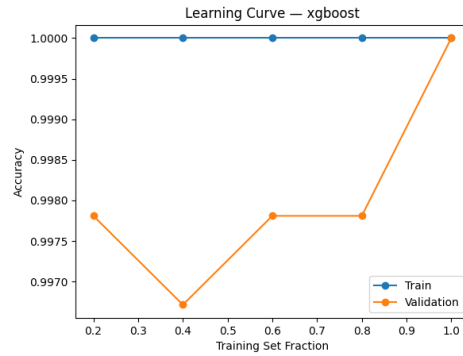


(b) UMAP

Figure 7: 2D embeddings of the validation feature space coloured by severity.



(a) Random Forest



(b) XGBoost

Figure 8: Learning curves (accuracy vs. training-set fraction).

5.12 Qualitative Inference Results

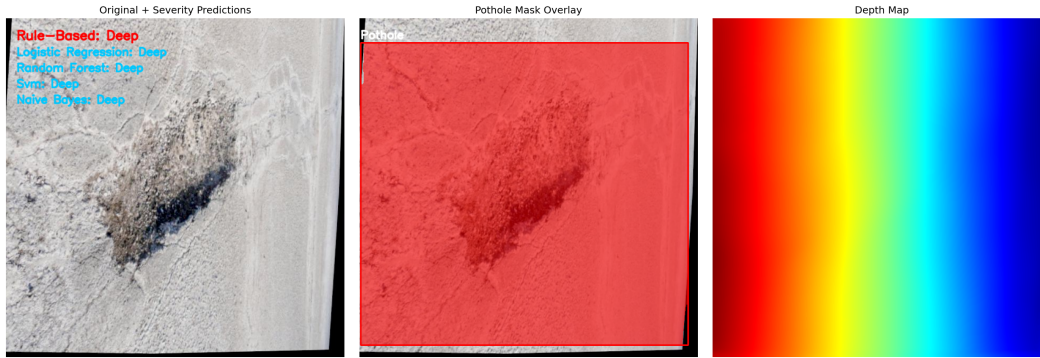
Figure 9 displays sample inference outputs. Each panel triplet shows: (i) the original image with overlaid severity predictions from all classifiers, (ii) the YOLOv8 segmentation mask with bounding box, and (iii) the Depth-Anything-V2 depth heatmap.

5.13 Accuracy Comparison (All Models)

Figure 10 presents a horizontal bar chart comparing accuracy across all nine classifiers. XGBoost, LightGBM, and the ensemble tie at the top, with Naïve Bayes trailing notably—a pattern consistent with the bootstrap analysis.



(a) Sample 1: Multiple potholes on a wet urban road.



(b) Sample 2: A large “Deep” pothole detected with strong consensus.

Figure 9: Qualitative inference results. Left: original image + classifier predictions. Centre: YOLOv8 segmentation mask overlay. Right: depth heatmap (Depth-Anything-V2).

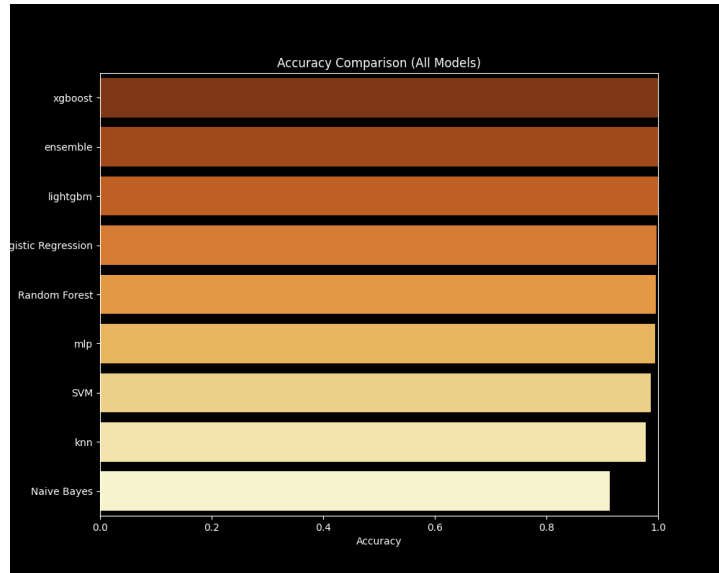


Figure 10: Accuracy comparison across all trained classifiers.

6 Web Application Demo

To make our pipeline accessible to non-technical stakeholders (e.g., municipal road inspectors), we have developed and deployed a full-stack web application named **Pothole Intelligence Studio**. The application is publicly hosted and accessible at:

<https://web-ui-sage-five.vercel.app/>

6.1 Architecture

- **Backend:** A FastAPI server exposes a `POST /analyze` endpoint that accepts image uploads, runs the complete inference pipeline (YOLOv8 + Depth-Anything-V2 + ML classifiers), and returns JSON results including base64-encoded visualization images. An `/insights/summary` endpoint serves pre-computed model performance metrics (classification reports, SHAP plots, confusion matrices) to the frontend. The backend is containerized via Docker and deployed on Hugging Face Spaces.
- **Frontend:** A React + Vite single-page application providing two views—*Detection* (image upload and result display) and *Model Insights* (performance dashboards). Deployed on Vercel with SPA routing.

6.2 User Interface

Figure 11 shows screenshots of the deployed application.

7 Limitations

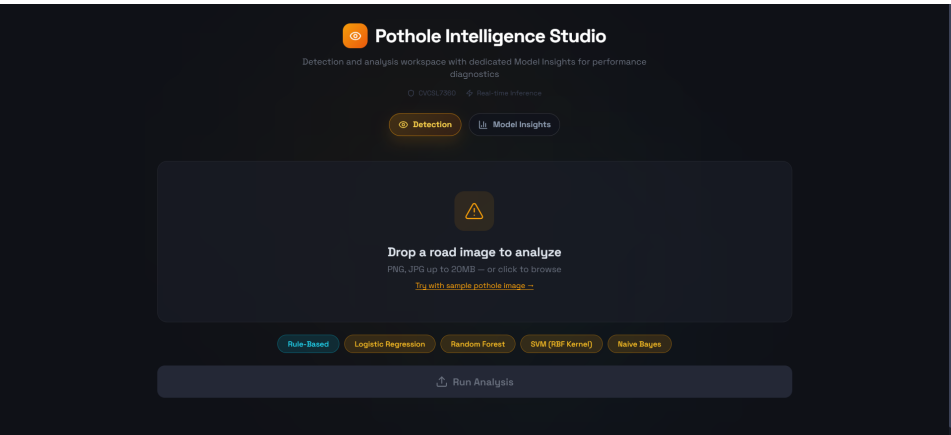
1. **Pseudo-label leakage.** Because KMeans pseudo-labels are derived from the same features (`max_depth`, `pothole_area`) that the classifiers use, the resulting high accuracy is somewhat inflated and does not necessarily reflect generalization to real-world severity ratings.
2. **Relative depth.** Depth-Anything-V2 produces relative (ordinal) depth, not metric depth. Consequently, the depth features are scale-ambiguous across different camera setups and distances. Per-image normalization mitigates this but also reduces cross-image discriminability.
3. **Dataset scope.** The current dataset (780 images) is relatively small and geographically limited. Performance on roads with significantly different textures, lighting, or camera angles is not guaranteed.
4. **Single-frame analysis.** The pipeline processes each image independently. A multi-frame or video-based temporal consistency module could improve robustness considerably.

8 Conclusions and Future Work

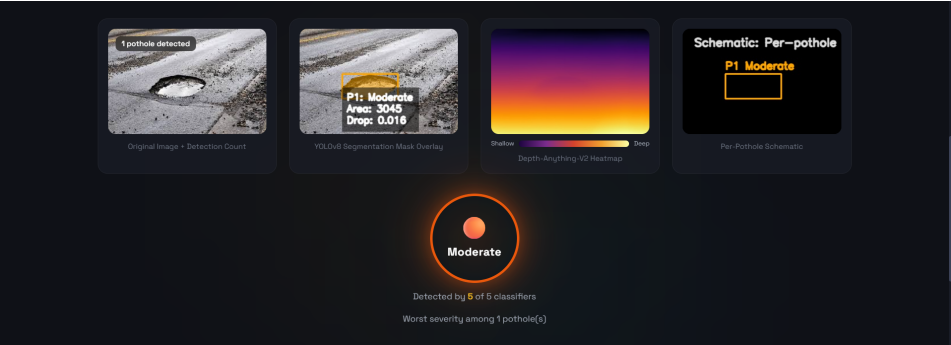
We have presented a comprehensive, modular pipeline for vision-based pothole detection and severity classification. By fusing YOLOv8 instance segmentation with Depth-Anything-V2 monocular depth estimation, we construct a rich 20-dimensional feature representation that captures both the geometry and the depth profile of each pothole. Nine classifiers—including gradient-boosted trees, neural networks, and a soft-voting ensemble—are trained and evaluated with bootstrap confidence intervals, SHAP-based explainability, ablation studies, and dimensionality reduction visualizations. The ensemble and gradient-boosted models achieve near-perfect accuracy on the current benchmark, and a publicly deployed web application makes the pipeline accessible for practical road maintenance workflows.

Looking ahead, several directions are promising:

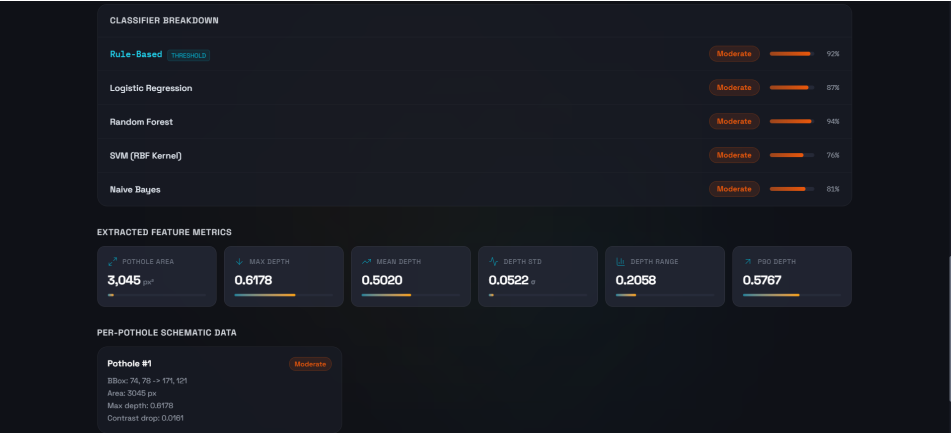
- Collecting and integrating *metric* ground-truth severity annotations (e.g., via LiDAR or structured-light sensors) to move beyond pseudo-labels.
- Exploring metric depth estimation models to produce absolute depth values, enabling volume-based severity metrics.
- Extending the pipeline to video streams with temporal tracking and frame-level consistency.



(a) Upload interface with classifier selection chips.



(b) Analysis output: original image, mask overlay, depth heatmap, per-pothole schematic, and consensus severity indicator.



(c) Classifier breakdown, extracted feature metrics, and per-pothole schematic data.

Figure 11: Screenshots of the *Pothole Intelligence Studio* web demo.

- Incorporating road segment-level aggregation (as prototyped in our `road_segment_analysis.py` module) to generate repair priority reports for entire road sections.
- Training a hybrid CNN classifier directly on image crops and depth patches, bypassing hand-crafted features altogether.
- Integrating standardized vehicle tyre size data with an AR-based absolute depth estimation approach to assess vehicle-specific pothole severity.

Acknowledgments

We thank Dr. Deepak Mishra, Dept. of CSE, IIT Jodhpur and the Course Instructor for CSL7360 : Computer Vision for his guidance and feedback throughout this project. We are grateful to the creators of the Pothole Image Segmentation Dataset on Kaggle and RDD Dataset for making their data publicly available. We also acknowledge the developers of Depth-Anything-V2, Ultralytics YOLOv8, scikit-learn, SHAP, and the broader open-source community whose tools made this work possible.

References

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. You only look once: Unified, real-time object detection. In *CVPR*, 2016.
- [2] Jocher, G., Chaurasia, A., & Qiu, J. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023.
- [3] Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., & Zhao, H. Depth Anything: Unleashing the power of large-scale unlabeled data. In *CVPR*, 2024.
- [4] Yang, L., Kang, B., Huang, Z., Zhao, Z., Xu, X., Feng, J., & Zhao, H. Depth Anything V2. *arXiv preprint arXiv:2406.09414*, 2024.
- [5] Nekouei, F. Pothole image segmentation dataset. <https://www.kaggle.com/datasets/farzadnekouei/pothole-image-segmentation-dataset>, 2023.
- [6] Sabri, G. Pothole segmentation and classification — YOLOv8 and Decision Trees. <https://github.com/gufransabri/Pothole-Segmentation-and-Classification-YoloV8-and-DecisionTrees>, 2024.
- [7] Koch, C., & Brilakis, I. Pothole detection in asphalt pavement images. *Advanced Engineering Informatics*, 25(3):507–515, 2011.
- [8] Maeda, H., Sekimoto, Y., Seto, T., Kashiya, T., & Omata, H. Road damage detection and classification using deep neural networks with smartphone images. *Computer-Aided Civil and Infrastructure Engineering*, 33(12):1127–1141, 2018.
- [9] Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., & Sekimoto, Y. Deep learning-based road damage detection and classification for multiple countries. *Automation in Construction*, 132:103935, 2021.
- [10] Fan, R., Ozgunalp, U., Hosking, B., Liu, M., & Pitas, I. Pothole detection based on disparity transformation and road surface modeling. *IEEE Transactions on Image Processing*, 29:897–908, 2019.
- [11] Ranftl, R., Lasinger, K., Hafner, D., Schindler, K., & Koltun, V. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE TPAMI*, 44(3):1623–1637, 2022.
- [12] Lundberg, S. M., & Lee, S.-I. A unified approach to interpreting model predictions. In *NeurIPS*, 2017.
- [13] Pedregosa, F., et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.
- [14] Chen, T., & Guestrin, C. XGBoost: A scalable tree boosting system. In *KDD*, 2016.
- [15] Ke, G., et al. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.